

Master build correlation

The three most recent saves, read against what this session built — and the finding that changes the integration plan

Read before `recommendations.md`. Every claim here was verified against the repository on 2026-09-18, and the command that produced it is named so it can be re-run.

0. The headline: there are two codebases, and one of them runs nowhere

	russell-capital-systems/	russell-capital/
TypeScript files	502	80
Shared engines	125	20
Test files	201	11
<code>package.json</code>	yes	no
<code>server/routers.ts</code>	yes	no
Imported by the deployed app	—	nothing imports it
Railway deploys it	yes	no

Verify: `comm -12 <(ls russell-capital/shared/.ts | xargs -n1 basename | sort) <(ls russell-capital-systems/shared/.ts | xargs -n1 basename | sort) and grep -rl "russell-capital/" russell-capital-systems/ (returns nothing).`

`russell-capital/` holds roughly **5,900 lines of finished, tested engine code that executes nowhere**. It has no build, no router, no route, and no importer. It is not dead code in the ordinary sense — it is good code in the wrong folder. Recovering it is the single highest-value integration task on the board, higher than anything in the previous roadmap, because the work is already paid for.

A second, smaller instance of the same problem sits inside the deployed folder: `russell-capital-systems/shared/timeMachine30.ts` (666 lines, with a 287-line test) is present, compiles, and appears in **no** route, **no** catalogue entry and **no** memory-bank group. Built, tested, invisible.

1. Save `482865c` — "engine kit, lending suite, scenario overlays, Time Machine"

2026-09-18 07:37 · 31 files · 10,566 insertions

The largest of the three. Twelve engines and eleven test files, almost all landing in `russell-capital/`.

File	Lines	My counterpart	Relationship	Action
realEstateMogul.ts	909	sequencePlanner.ts	Complementary — the layer beneath mine. It does per-loan amortisation (amortize), lostPurchasingPower, analyzeLoan, analyzeProperty, summarizePortfolio up to MAX_PROPERTIES = 150, and threading. My planner decides <i>which move next</i> ; this decides <i>what a property and its loans are actually doing</i> . My tick() does a crude 45%-haircut paydown where this has a real schedule.	Port and adopt. Replace the planner's internal amortisation with amortize(); feed analyzeProperty into each Stage.after.
sequenceStress.ts	616	upgrade branch 2 (adverse replay)	This IS the branch I proposed, already written. mulberry32 seeded RNG, resequence, runSequence, sequenceStressTest with MAX_SHUFFLES = 20, policySurvivalMonteCarlo, SurvivalResult. I recommended building it; it exists.	Port first. Wire sequenceStressTest to take a Plan from my planner. This closes the largest risk-mitigation gap in one move.
timeMachineCompliance.ts	947	(nothing)	Pure gain. Illustration/AG-49 compliance logic the deployed app has no equivalent of.	Port, then gate every illustration surface behind it.
nextBestAction.ts	560	genomeStrategyFits	Direct competitor. Lens-based scoring: LENSES, LENS_WEIGHTS by RiskAppetite, scoreThroughLenses, rankStrategies, TransferRisk. Mine uses signals/gates with silence≠evidence. Two ranking engines cannot both be authoritative.	Reconcile, do not merge blindly. See §5.
scenarioEngine.ts	402	sequenceArchetypes.ts	Overlap — both hold named scenarios.	Diff, keep one, port the other's content as data.
factFinder.ts	321	clientFactFinder.ts	Overlap.	Reconcile into the single Situation object (roadmap task 1.9).
helocLenders.ts	239	thresholds.ts#heloc-cltv + velocity providers	Complementary and better in one respect: it defines UnsourcedQuoteError — an unsourced quote <i>throws</i> — plus rankLenders, rotationCapacity, LENDER_RULES.	Port. Adopt the error class everywhere (see §4).

File	Lines	My counterpart	Relationship	Action
divorceStateRules.ts	244	householdGenome.ts	Complementary — state rules the pairing protocol should consult.	Port and wire into household genome.
smallBusinessLending/{industryRisk, loanPricing, prospecting}.ts	1,422	(nothing)	Pure gain. An entire vertical the deployed app lacks.	Port as a new page group.
patentCatalog.ts / patentStatus.ts	195 / 117	same names, 237 / 161 lines	Diverged duplicates. The systems versions are larger.	Diff line by line; keep the systems version unless the smaller one holds a claim the larger lacks.
policyLoanMechanics.ts	325	same name, 405 lines	Diverged duplicate.	Same treatment.
timeMachine30.ts + pacificHorizonEcv.ts	666 + 38	—	Landed in russell-capital-systems/ but unrouted, uncatalogued, unwired to the brain.	Route it, catalogue it, add to a memory group. One hour of work for a finished engine.

2. Save 5e3c1ec — "multiplier decision engine, 80% loan strategy, decision page"

2026-09-18 14:58 · 6 files · 2,302 insertions

File	Lines	My counterpart	Relationship	Action
multiplierDecision.ts	626	policyLoanMechanics.ts, my policy:borrow stage	Complementary — decision logic for multiplier/bonus products.	Port; let it supply the planner's policy stage numbers.
annualLoanStrategy.ts	392	my policy:borrow move	Directly relevant. An 80%-loan annual strategy is exactly the mechanic my planner models crudely (bestVariant on policy-loan-value).	Port and have the planner call it instead of its own arithmetic.
indexCreditingData.ts	685	same name, 999 lines in systems	Diverged duplicate.	Diff; the systems version is larger but the smaller may be newer. Check content, not size.
showcase/multiplier-decision.html	327	—	A standalone showcase page.	Convert to a portal route or delete; a showcase HTML outside the router is invisible.

3. Save 7ab685c — "the council, the NLP layer, guarded health evidence, Patent360 audit"

2026-09-18 15:10 · 6 files · 1,556 insertions

This is the save that overlaps my work most directly, and it contains the single best idea in the three saves.

File	Lines	My counterpart	Relationship	Action
council/aiCouncil.ts	443	compositeMind.ts + aiMemoryBank.ts	Direct overlap, and theirs is stronger on one axis. COUNCIL members by Domain, classifyIntent, routeUtterance, RoutingMemory, convene, CouncilAnswer, Finding — and UnsourcedFindingError.	See §4. Reconcile the roster; adopt the error class immediately.
council/nlpEngine.ts	324	nlpBrain.ts	Overlap.	Diff and merge; mine has 51 meta-programs and 32 patterns.
health/evidenceRetrieval.ts	286	provenance.ts	Complementary — the retrieval half of provenance. Mine traces a figure that has been computed; this retrieves the evidence behind a claim. Together they close the loop.	Port and pair with FigureTrace.
docs/PATENT360_AUDIT.md	144	patentStatus.ts guard	An audit of the patent position.	Read before filing; reconcile with the 57-claim count.

4. The best idea in the three saves, and it is not mine

aiCouncil.ts defines UnsourcedFindingError. helocLenders.ts defines UnsourcedQuoteError. Both make "no figure without a source" a **runtime type error that throws**, rather than a rule written in prose and enforced by a test that scans source text.

My approach across thresholds.ts, mechanismDossiers.ts and aiMemoryBank.ts was the prose-and-guard-test approach: verified<T> makes the *shape* impossible to get wrong, and tests scan for phone-shaped strings and forbidden phrases. That is good, and it catches a class of error at build time. But it does not stop a figure computed at runtime from reaching a page without provenance, because no test can scan a value that did not exist when the test ran.

The other session's pattern is strictly stronger and should become the house standard. Recommendation, in order:

1. Move UnsourcedFindingError into a shared module — shared/sourcing.ts — alongside Verified<T>.
2. Add assertSourced<T>(value, source, asOf): Verified<T> that throws UnsourcedFindingError when source or asOf is absent.
3. Have every engine that emits a client-visible number route it through assertSourced. Start with the twelve tRPC routers that fetch external data, because those are the ones whose values are born at runtime.
4. Keep the existing guard tests. Belt and braces: the type makes it hard, the test makes it visible, the error makes it impossible.

This is the clearest case in the whole correlation of the two codebases being better together than either alone.

5. The one genuine conflict: two ranking engines

`nextBestAction.ts` (rankStrategies through weighted lenses by risk appetite) and `genomeStrategyFit.ts` (signals and gates, where an unanswered factor lowers confidence rather than raising a score) both rank strategies for a household. **They will disagree, and both will be on the site.**

Do not merge them by averaging. The honest resolution, in preference order:

- **Option A — one ranks, one explains (recommended).** `genomeStrategyFit` decides eligibility and confidence; `nextBestAction`'s lenses become the *explanation* of why a strategy ranks where it does. Lens scores render as reasons, not as a competing order. One authority, richer output.
- **Option B — run both, show the disagreement.** Present both rankings side by side wherever they differ and say why. Honest, and genuinely differentiating: no competitor shows a household two rankings and the reason they differ. Costs more UI.
- **Option C — replace mine.** Only if `nextBestAction`'s lens weights are better evidenced than my signals/gates. Check whether the weights are sourced or chosen; mine are chosen, and I said so.

Do not ship both as authoritative. Two numbers that both claim to be the answer is the failure mode this whole system is built to avoid.

6. Integration order — what to do, in what sequence

Each step is independently shippable and leaves the suite green.

Step	Work	Why this order	Done when
1	Route, catalogue and brain-wire <code>timeMachine30.ts</code>	Finished engine, zero risk, one hour, proves the pipeline	<code>/portal/time-machine-30</code> live; scorecard wiring ≥ 8
2	Create <code>shared/sourcing.ts</code> with <code>UnsourcedFindingError</code> + <code>assertSourced</code>	Everything after it should be built on the stronger pattern	All twelve data routers route through it; a test asserts a source-less value throws
3	Port <code>sequenceStress.ts</code> , wire to <code>Plan</code>	Closes the biggest risk gap; the code exists	Every plan shows its break year under all four recorded shocks
4	Port <code>realEstateMogul.ts</code> ; replace the planner's internal amortisation	Makes every stage figure real rather than approximated	<code>Stage.after</code> carries a real schedule; existing planner tests still pass
5	Diff and resolve the four duplicate modules	Must happen before more code depends on either copy	One copy of each; the loser deleted, not left to rot
6	Reconcile the two ranking engines per §5 Option A	The most user-visible inconsistency	One authoritative order, lens scores as explanation
7	Port <code>helocLenders</code> , <code>divorceStateRules</code> , <code>evidenceRetrieval</code> , <code>timeMachineCompliance</code>	Pure additions, no conflicts	Each routed, catalogued, tested, brain-wired
8	Port <code>smallBusinessLending/*</code> as a new page group	Largest new surface; do it once the pattern is settled	Three pages live with the full wiring
9	Merge <code>nlpEngine</code> into <code>nlpBrain</code> ; reconcile <code>aiCouncil</code> with <code>compositeMind</code>	Highest-risk merge, do it last with the pattern proven	One roster, one voice, tests green
10	Delete <code>russell-capital/</code> or convert it to a documented staging folder with a README saying it is not deployed	An orphan folder that looks live is how this happened	Either gone, or labelled

Step 10 matters more than its position suggests. The root cause of 5,900 orphaned lines is a folder that looks like an app and is not one. Fixing the symptom without fixing the cause means doing this again in a month.

7. What I would tell the next builder in one paragraph

The deployed app is `russell-capital-systems/`. Everything else is staging. Before writing a new engine, search both folders for its name — four modules already exist twice and have diverged. The strongest pattern in the codebase is not mine: it is `UnsourcedFindingError`, and it should be everywhere. The biggest immediate win is not new code at all — it is routing `timeMachine30`, porting `sequenceStress`, and adopting `realEstateMogul`'s amortisation, all of which are finished and tested and currently execute nowhere. And the integration scorecard at `/portal/integration-scorecard` will tell you, every time you load it, whether what you just did actually connected to anything.